

Projeto Aplicado II - Etapa 3

Integrantes:

Alexandre Zamarion Cepeda
Felipe Gabriel Lopes Pinheiro Clarindo
Thiago Godeguesi
Victória Martins Fortes

Professor:

Felipe Albino dos Santos

1. Premissas do Projeto

Organização:

A empresa fictícia criada será denominada Internet Movies DataBase Analytics (IMDb) , especializada em soluções de análise preditiva voltadas para filmes.

Área de Atuação: A IMDb atua no setor de recomendações de filmes, com ênfase em modelos de aprendizado de máquina para entender a recomendação dos usuários, prever a classificação dos melhores filmes segundo a classificação, entender o sentimento referente aos filmes e a classificação dos usuários.

Tipo de dado: texto (reviews de filmes em linguagem natural, com rótulo binário *positive/negative*).

Base de Dados Utilizada:

- Nome: IMDb Dataset of 50K Movie Reviews.
- Fonte: Kaggle (Lakshmi N). - Formato: CSV.
- Tamanho: ~50.000 registros dos usuários.
- Contexto: Dados de review dos filmes, feito por usuários, juntamente com os sentimento das reviews dos usuários.

Link: [Kaggle](#)

Metadados relevantes:

- Review
- Sentiment

2. Estrutura dos Dados (principais variáveis) e análise exploratória

Variável	Descrição
review	Texto com a review do usuário
sentiment	Informação se o sentimento é positivo ou negativo.

Dataset:

Carregamos o dataset bruto no Colab e verificamos sua estrutura: 50.000 linhas e 2 colunas (*text* e *label*). A amostra das primeiras linhas confirma que a coluna *text* contém os reviews em linguagem natural e *label* traz o rótulo de sentimento (por exemplo, “positive”).

Observamos também a presença de marcação HTML (como `<br />`) em alguns textos, indicando a necessidade de limpeza na etapa de preparação.

Durante a preparação, removemos essas marcações `<br>` (incluindo `<br/>` e `<br />`) e colapsamos espaços extras; após a limpeza, não restaram ocorrências de `<br>` no campo *text*.

Por fim, a checagem de completude mostrou ausência de valores nulos em ambas as colunas (*text*: 0, *label*: 0), o que facilita o pré-processamento e a análise exploratória subsequente

Duplicatas:

Procuramos por duplicatas exatas por (*text*, *label*) e imprimimos alguns exemplos para inspeção. Em seguida, removemos duplicatas exatas para evitar contagem dupla de exemplos idênticos e auditamos casos de mesmo texto com rótulos conflitantes, e nenhum caso foi encontrado.

Esse processo foi executado antes da divisão treino/validação para mitigar vazamento de dados.

Resultado: identificamos 824 linhas pertencentes a 406 grupos de duplicatas exatas por (*text*, *label*); mantivemos uma ocorrência por grupo e removemos 418 linhas duplicadas, antes do *split* (alguns grupos tinham mais de duas ocorrências).

Validação do schema:

Verificamos o contrato de dados garantindo que a coluna *text* seja do tipo string em 100% das linhas e que o *label* contenha apenas os valores esperados (positivo, negativo).

A base resultante após a limpeza possui 49.582 registros, com distribuição de classes próxima do balanceamento: 24.884 positivos e 24.698 negativos. Esses cheques asseguram consistência estrutural antes do *split* e da modelagem.

Distribuição de classes:

Após a limpeza, a base apresenta 24.884 avaliações positivas (50,19%) e 24.698 negativos (49,81%), ou seja, distribuição praticamente balanceada entre as classes.

Esse equilíbrio reduz a necessidade de técnicas de reamostragem e permite usar métricas padrão (F1 como principal, além de Precisão e Recall) com boa interpretabilidade.

Mesmo assim, adotaremos divisão estratificada no *split* de treino/validação para preservar essa proporção em todos os subconjuntos.

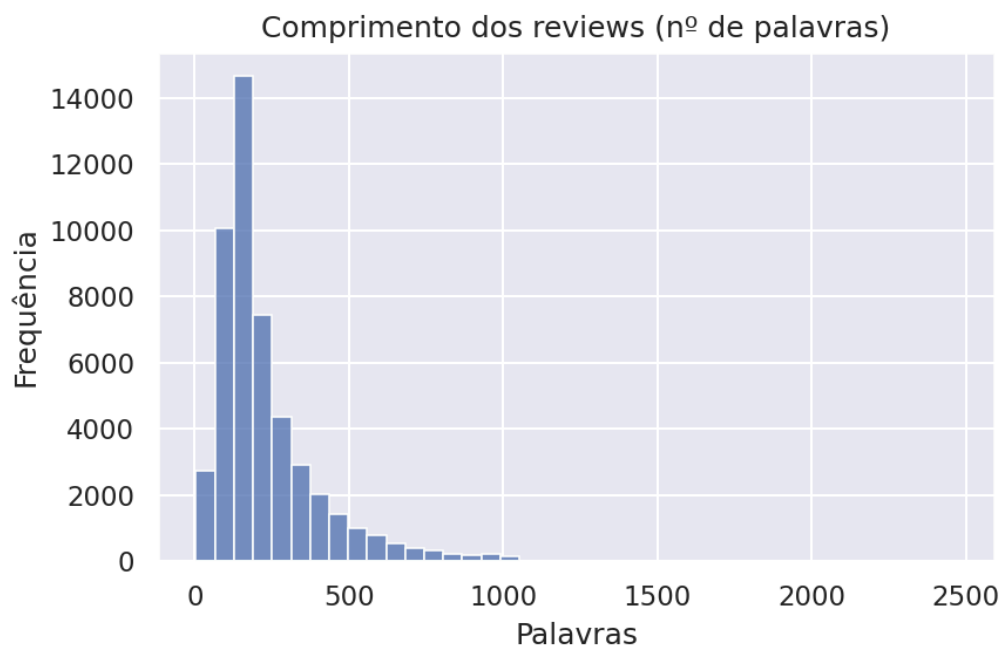
Comprimento do texto:

Considerando 49.582 reviews, o comprimento médio é de 231 palavras (desvio-padrão ≈ 172), com mediana de 173 palavras (Q1=126, Q3=281).

Há desde textos muito curtos (mínimo=4 palavras) até muito longos (máximo=2.470 palavras), indicando alta variabilidade no tamanho dos reviews.

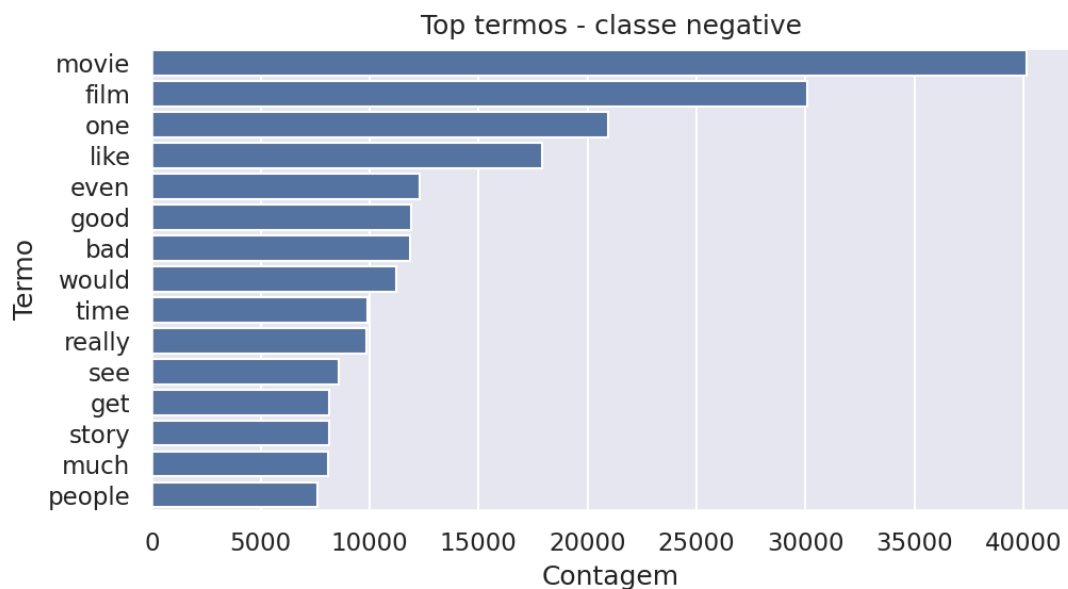
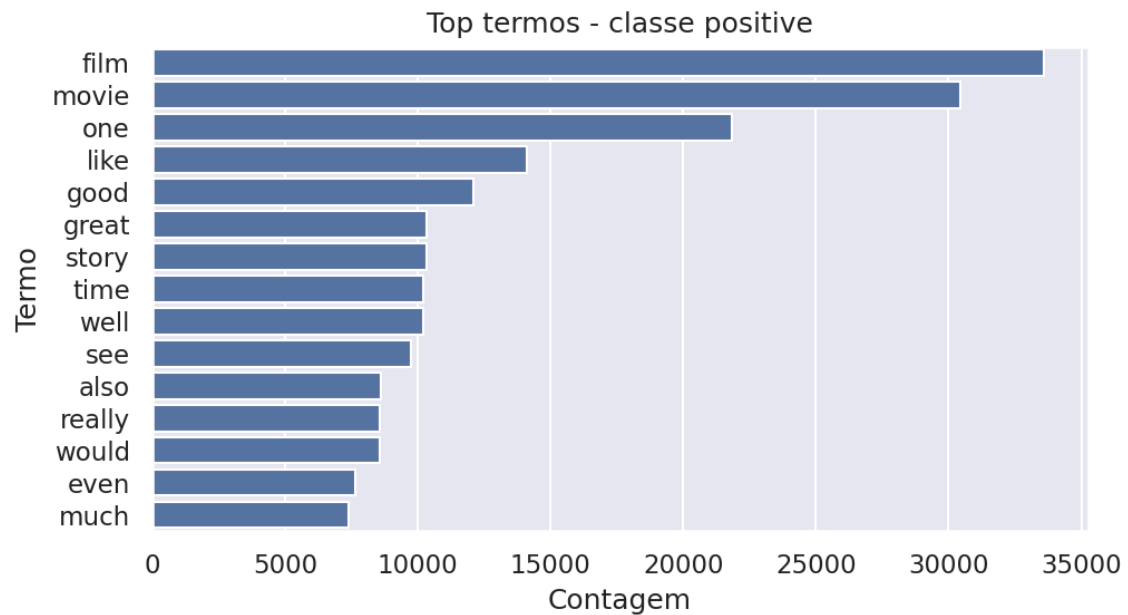
Essa dispersão reforça a necessidade de normalização/limpeza e justifica o uso de representações como TF-IDF, que lidam bem com documentos de comprimentos distintos.

Para treinamento, adotaremos divisão estratificada preservando a proporção de classes, e manteremos os parâmetros do vetorizador (ex.: `min_df`) documentados para mitigar termos raríssimos oriundos de textos muito curtos/longos.



Top termos positivos e negativos:

Abaixo temos a representação da frequência dos termos que mais apareceram nos reviews positivos e negativos.



3. Resultados, acurácia e esboço do storytelling

Método aplicado:

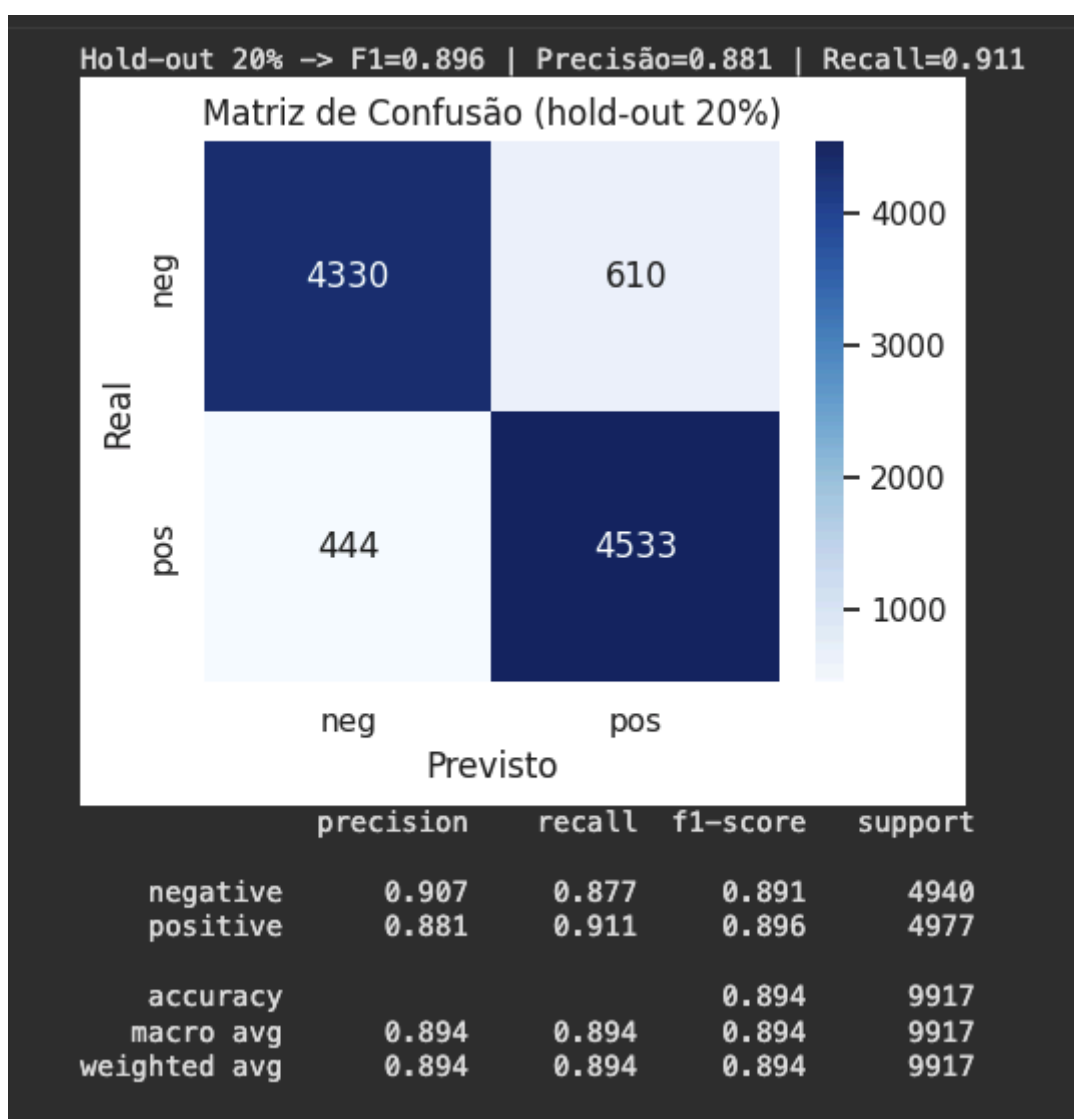
Aplicamos o pipeline da Etapa 2: TF-IDF (unigramas; stopwords em inglês; min_df=5) + Regressão Logística (L2). Fizemos split estratificado 80/20 e, no conjunto de treino, validação cruzada estratificada (k=5) com GridSearch em $C \in \{0.1, 1, 10\}$.

Resultado da validação: Melhor ajuste: C=1, com F1 médio (CV) $\approx 0,893$.

Desempenho no hold-out (20%). F1 = 0,896, Precisão = 0,881, Recall = 0,911 e Acurácia = 0,894. A matriz de confusão (TN=4330, FP=610, FN=444, TP=4533) é consistente com a validação e confirma o baseline com leve ênfase em recuperar positivos.

Em nosso repositório do GitHub entregamos um classificador de sentimento pronto para uso, além de outros gráficos e tabelas.

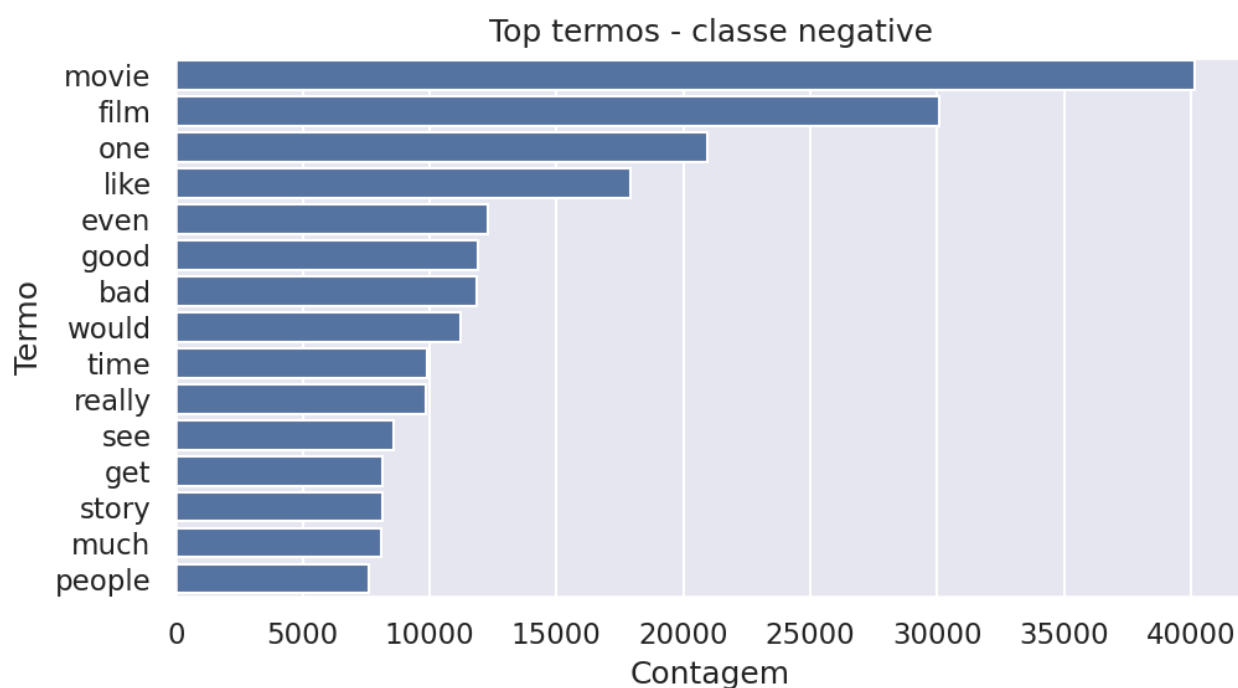
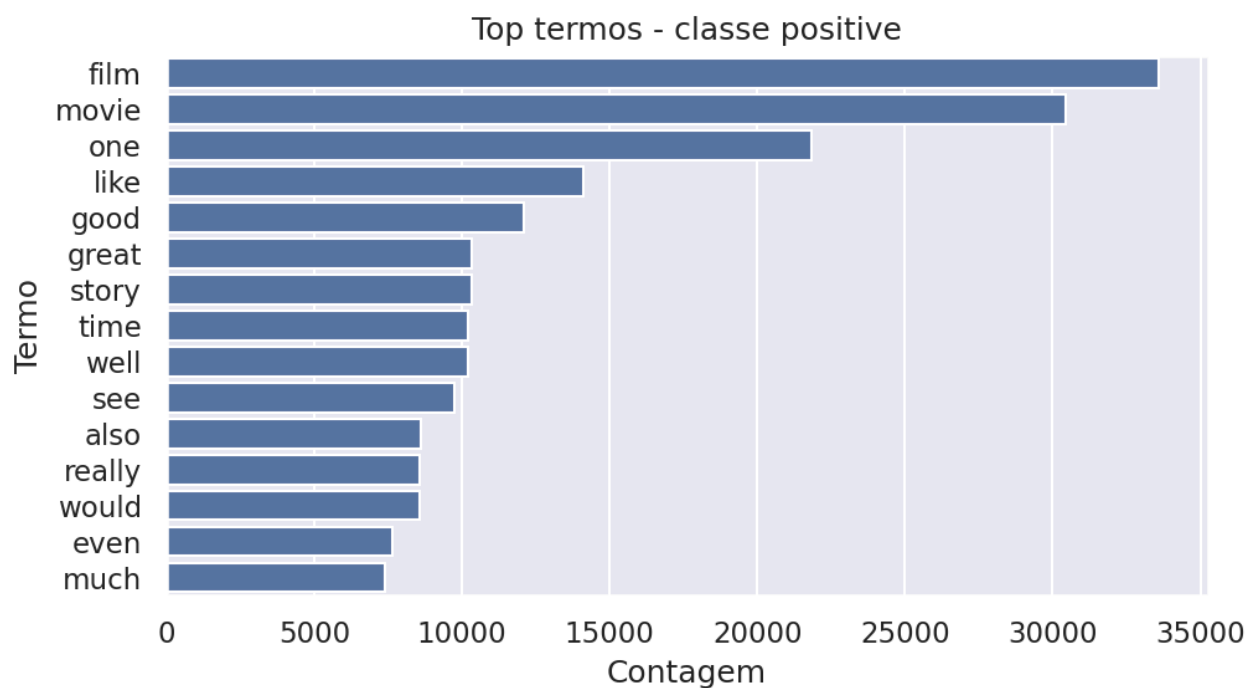
Figura: Matriz de Confusão e Tabela 1 — Métricas no hold-out



Interpretabilidade (termos influentes)

A partir dos coeficientes da Regressão Logística sobre TF-IDF, listamos os 20 termos que mais puxam a predição para positivo e para negativo. Os coeficientes positivos aumentam a chance de *positivo* e os negativos aumentam a chance de *negativo*.

Figura: Termos mais associados a positivo e termos mais associados a negativo



Medidas de acurácia:

Usamos F1 como métrica principal, por equilibrar Precisão e Recall em classificação binária. A seleção do modelo foi feita pelo F1 médio nos 5 folds estratificados do treino. No conjunto de validação (20%), reportamos F1, Precisão e Recall e apresentamos a matriz de confusão para interpretar erros (FP/FN).

O modelo priorizou recuperar positivos (Recall=0,911), aceitando um pouco mais de falsos positivos (Precisão=0,881)

Produto:

Entregamos um classificador de sentimento que recebe um texto e retorna positivo/negativo com pontuação de confiança. O modelo completo (TF-IDF + LogReg) foi salvo em nosso repositório do GitHub (logreg_tfidf_pipeline.pkl), permitindo uso posterior sem re-treino. Para explicabilidade leve, apresentamos também em nosso repositório os 20 termos mais associados a cada classe (pos/neg), úteis para auditoria e comunicação.

Modelo de negócio:

Público-alvo: equipes de atendimento/marketing e plataformas com grande volume de reviews. Proposta de valor: monitorar reputação, priorizar críticas relevantes e identificar tendências por sentimento. Entrega: API simples para classificação em lote e um mini-dashboard com volume por classe e alertas semanais. Monetização: camada gratuita limitada; plano Pro com histórico completo, exportação e integrações.

Esboço do storytelling

- Problema & contexto: por que ler milhares de reviews manualmente é inviável.
- Dados & qualidade: IMDb 50k, limpeza (
 removido), deduplicação e distribuição de classes equilibrada.
- Abordagem: TF-IDF + Regressão Logística; validação k=5; por que essa escolha.
- Resultados-chave: F1/Precisão/Recall; Matriz de Confusão; 1 padrão de erro (ironia/negação).
- Produto (demo): “Cole o review → sentimento + confiança” (mock).
- Valor & negócio: quem usa, como captura valor, sketch de monetização.
- Como próximos passos, avaliaremos bigrams e tratamento de negações para capturar melhor contextos como ‘not good’, ajustaremos min_df para equilibrar ruído e cobertura, compararemos com SVM linear como baseline alternativo e, opcionalmente, reportaremos PR-AUC para complementar F1.

Reprodutibilidade e artefatos

Usamos `random_state=42` em `split` e `CV`; versões de bibliotecas estão no notebook. Artefatos: métricas/figuras e modelo em `estão` salvos no repositório. Isso garante que os resultados possam ser reproduzidos e auditados.

4. Objetivos e Metas

Objetivo Geral:

Construir um classificador de sentimento para reviews do IMDb e produzir insights textuais (ex.: termos mais influentes e nuvem de palavras).

Objetivos Específicos:

- Coletar e organizar a base (IMDb 50K), registrando fonte e licença.
- Fazer EDA básica: balanceamento de classes, tamanho dos textos e exemplos.
- Limpar e preparar o texto: lowercase, remoção de HTML/URLs/stopwords e tokenização.
- Representar o texto com TF-IDF (unigramas).
- Treinar Regressão Logística (baseline).
- Avaliar por F1 (principal) + Precisão e Recall.
- Entregar relatório técnico curto com achados e próximos passos.
- Tipo de dado: texto (reviews de filmes em linguagem natural, com rótulo binário *positive/negative*).

Metas Temporais:

- Curto prazo (30 dias): Preparação e análise exploratória.
- Médio prazo (60 dias): Treinamento e avaliação de modelos.
- Longo prazo (90 dias): Relatório técnico final e painel de resultados.

5. Cronograma de Atividades (Estimado)

Fase	Atividade	Período	Responsáveis
1	Definição do grupo, empresa e dataset	Semana 1–2	Todos
2	Coleta, limpeza e preparação dos dados	Semana 3–4	Alexandre e Felipe
3	Análise exploratória (EDA)	Semana 5–6	Thiago
4	Implementação dos modelos preditivos	Semana 7–8	Victória
5	Avaliação dos modelos e métricas	Semana 9–10	Todos

6	Relatório técnico e documentação no GitHub	Semana 11–12	Todos
7	Apresentação final	Semana 13	Todos

6. Estrutura do Documento

Este documento contempla: grupo de trabalho, premissas do projeto, estrutura dos dados, objetivos e metas, cronograma de atividades

7. Link para o GitHub e bibliotecas utilizadas

[GitHub](#)

Pretendemos utilizar inicialmente as bibliotecas Python:

- **pandas**: com **pandas**, carregaremos os dados, faremos limpeza básica e organizaremos tabelas (dataframes) para EDA e modelagem.
- **numpy**: com **numpy**, realizaremos operações numéricas eficientes (vetores/matrizes) que dão suporte ao pré-processamento e às representações dos dados.
- **nltk**: com **nltk**, removeremos **stopwords** e aplicaremos **tokenização simples** para preparar o texto antes da vetorização.
- **matplotlib**: com **matplotlib**, criaremos **gráficos** (distribuição de classes, comprimento dos textos, resultados) para visualizar EDA e métricas.
- **scikit-learn**: com **scikit-learn**, representaremos os reviews com TF-IDF, treinaremos um classificador e avaliaremos o desempenho com F1, Precisão e Recall.
- **seaborn**: criaremos gráficos estatísticos prontos para agilizar a EDA e a visualização da matriz de confusão.

8. Bases teóricas (TF-IDF e Regressão Logística)

TF-IDF (representação de texto):

Transformaremos cada review em um vetor numérico cujas “features” são termos do vocabulário.

O peso de cada termo combina **TF** (frequência do termo no documento) e **IDF** (inverso da frequência do termo no corpus): termos frequentes no review e raros no conjunto recebem peso maior. Isso reduz a influência de palavras muito comuns (ex.: “the”) e destaca sinais informativos (ex.: “excellent”, “awful”), produzindo vetores esparsos e adequados para modelos lineares.

Regressão Logística (classificação binária):

É um modelo discriminativo que estima a probabilidade de uma classe (p.ex., *positivo*) por meio da função logística aplicada a uma combinação linear das features (os pesos do vetor TF-IDF).

Por lidar bem com alta dimensionalidade e esparsidade, costuma ter ótimo desempenho em NLP clássico, treina rápido e permite interpretação dos coeficientes (termos que aumentam ou diminuem a chance de *positivo*).

Usaremos regularização (padrão L2) para evitar sobreajuste. Em texto temos muitas features (milhares de palavras) e várias são raras. Sem controle, a Regressão Logística pode dar peso exagerado a termos pouco representativos.

A L2 distribui melhor a importância entre os termos, reduz variância e evita que um termo isolado “domine” a predição.

Treinamento (pipeline):

Vamos transformar os textos em números usando TF-IDF (palavras isoladas), ignorando stopwords em inglês e descartando termos muito raros (aparecem em menos de 5 reviews). Em seguida, vamos treinar uma Regressão Logística com um “freio” para evitar exageros nos pesos (regularização L2).

Dividiremos os dados em 80% para treinar e 20% para validar, preservando a proporção de positivos/negativos. Durante o treino, testaremos três forças desse freio ($C = 0,1; 1; 10$) e ficaremos com a melhor segundo a métrica F1. No final, avaliaremos o modelo no conjunto de validação usando F1 (principal), Precisão, Recall e a matriz de confusão (para ver acertos e erros).

9. Acurácia:

A acurácia do trabalho será reportada por F1-score (métrica principal), com Precisão e Recall complementares. Durante o ajuste, usaremos validação cruzada estratificada ($k=5$) no treino e escolheremos o melhor modelo pelo F1 médio. No conjunto de validação (hold-out 20%), reportaremos F1, Precisão, Recall e a matriz de confusão.

Reprodutibilidade: fixaremos `random_state = 42` em todas as etapas (split e treino).

10. Justificativa Acadêmica e Profissional

A escolha do IMDb 50K Movie Reviews é adequada para NLP: permite comparar representações textuais (Bag-of-Words/TF-IDF/embeddings) e classificadores supervisionados (Regressão Logística/SVM), avaliando F1/Recall/Precisão em um problema clássico de análise de sentimentos.

Profissionalmente, esse tipo de modelo apoia monitoramento de percepção, curadoria editorial e sistemas de recomendação em plataformas de mídia.

Do ponto de vista acadêmico, o dataset possibilita explorar técnicas de:

- Estatística descritiva de texto: distribuição de classes, comprimento dos reviews, vocabulário e n-grams.

- NLP supervisionado (classificação de sentimento): treinamento e comparação de modelos.
- Representações textuais: Bag-of-Words e TF-IDF (opcional: embeddings).
- Pré-processamento linguístico: limpeza (lowercase, remoção de HTML/URLs/stopwords) e tokenização.
- Avaliação de desempenho: F1 (principal), Precisão, Recall e matriz de confusão (opcional: PR-AUC).
- Relato técnico e comunicação dos resultados (storytelling dos achados).